

Transformation Trait **uncvref**

Document #: WG21 P0550R1
Date: 2017-06-11
Project: JTC1.22.32 Programming Language C++
Audience: LEWG $\Rightarrow$ LWG
Reply to: Walter E. Brown <webrown.cpp@gmail.com>

Contents

1	Introduction	1	5	Proposed wording	4
2	Discussion	2	6	Acknowledgments	4
3	Naming	3	7	Bibliography	5
4	Further discussion	3	8	Document history	5

Abstract

This paper proposes **uncvref**, a new TransformationTrait for the `<type_traits>` header. Like **decay**, **uncvref** removes any *cv* and reference qualifiers, but unlike **decay**, it does not mimic any array-to-pointer or function-to-pointer conversion.

I love criticism just so long as it's unqualified praise.

— NOEL COWARD

I'm unqualified for anything else. I'm barely qualified for this.

— CATHERINE KEENER

Knowledge unqualified is knowledge simply of something learned.

— PLATO

1 Introduction

The **decay** trait¹ sees significant use in the Standard Library, as it plays an important role in the specification of several Library components. Alas, a number of these uses are more elaborate than is strictly necessary. In particular, **decay** is used in several places where it would suffice simply to strip *cv* and reference qualifiers, with no need for the decaying conversions (array-to-pointer and function-to-pointer) that gave rise to **decay**'s name.

To address this situation, this paper proposes² **uncvref**, a new TransformationTrait³ for the `<type_traits>` header. Like the **decay** trait, **uncvref** removes any *cv* and reference qualifiers; unlike **decay**, it does not mimic any array-to-pointer⁴ or function-to-pointer⁵ conversion. The next section will provide several examples in which this proposed trait would serve as a more accurate and slightly cheaper alternative to the current use of **decay**.

Copyright © 2017 by Walter E. Brown. All rights reserved.

¹See [N4659], [meta.trans.other].

²This proposed trait has previously been the subject of Library issue 1120, closed as NAD Future in 2009-10.

³See [N4659], [meta.rqmts]/3.

⁴See [N4659], [conv.array].

⁵See [N4659], [conv.func].

2 Discussion

To a first approximation, the full `decay` treatment seems typically needed when forwarding arguments. In contrast, merely comparing types seems typically to require only `uncvref`. In the absence of the latter (proposed) trait, `decay` has frequently served as a convenient substitute for it. Here are several examples (from [N4659]) of such overly enthusiastic use of `decay`:

- The following excerpt occurs twice in [tuple.apply]:⁶

```
... make_index_sequence<tuple_size_v<decay_t<Tuple>>>{}>();
```

In this context, the `decay_t` metafunction call is unrelated to any possible array or function type. Instead, it is only the unqualified `Tuple` type that is wanted, so a call to the proposed `uncvref` would suffice:⁷

```
... make_index_sequence<tuple_size_v<uncvref_t<Tuple>>>{}>();
```

- As a more interesting example, consider [optional.ctor]/23, where we find the metafunction calls:

```
is_same_v<decay_t<U>, in_place_t>
...
is_same_v<optional<T>, decay_t<U>>
```

As before, there is no role here for any decay conversion; only *cv* and reference qualifiers need be removed from type `U`. These therefore seem perfect candidates for the proposed `uncvref` trait, leading to:

```
is_same_v<uncvref_t<U>, in_place_t>
...
is_same_v<optional<T>, uncvref_t<U>>
```

instead. (A similar example is found in [variant.ctor]/16.)

- Finally, in [func.require], we find several conditions asking about the relationship of `decay_t<decltype(t1)>` to other types (e.g., to `reference_wrapper`). Applying `uncvref` instead of `decay` would suffice for these, too.⁸

In all, the Library clauses directly apply `decay_t` circa forty times; we recommend that each be audited along the lines we have begun above. If the present proposal is accepted, we are prepared to undertake these audits and report their result, with recommendations, in a future paper. (Clause 30 also uses the macro-like `DECAY_COPY` almost twenty additional times; these, too, should be similarly audited, although Lavavej opines⁹ “that every use of `DECAY_COPY` is necessary.”)

Finally, we note that several vendors have already implemented the proposed trait, under various private names. The next section will discuss the proposed trait’s name.

⁶A previous Working Draft actually contained a third instance of just such an excerpt, but the Example (in [intseq.make]) of which it was a part has been editorially removed since that Draft was published.

⁷Even that much is technically unnecessary; since `tuple_size` is defined for *cv*-qualified types, it would here suffice to remove reference qualification, leaving any *cv*-qualification.

⁸Similarly, applying `uncvref` instead of `decay` would suffice in the [futures.task.members]/3 specification that currently reads “... if `decay_t<F>` is the same type as `packaged_task<R(ArgTypes...)>`.”

⁹Stephan T. Lavavej: “Re: remove_cv_ref.” Personal correspondence, 2017-01-05.

3 Naming

From a number of private conversations regarding the proposed new trait's name, the following candidates have emerged:

- **`remove_const_volatile_reference`**: There was widespread agreement that this is the most descriptive name. If it were shorter, it would be the obvious choice, but no one wanted this much to type.
- **`remove_cv_reference`**: Even this was seen as too long a name.
- **`remove_cv_ref`**: This seemed acceptable to all parties, but rather grudgingly so. No one wanted to lobby for it very strongly. One individual did rate the variant spelling **`remove_cvref`** as slightly more preferable for reasons of consistency with existing **`remove_*`** traits.
- Nicolai Josuttis wrote in a discussion thread (see §4) that he “strongly suggest[s] to name the trait **`remove_ref_cv`** instead of **`remove_cv_ref`** because it is important that first the reference is removed.”
- **`strip`**: This name is in private use for the trait; it had been chosen by that implementer because the trait “strips qualifiers” from the given type. The name was generally seen as acceptable, but without significant enthusiasm due mostly to a lack of specificity.
- In Library issue 1120, the proposed name was **`remove_all`**. This name was at the time seen as “too generic” with “a possible alternative matching the current naming style could be **`remove_cv_reference`** or **`remove_reference_cv`**.” Further, “it might be easier to cho[o]se the name not in terms of what it removes (which might be a lot), but in terms of [what] it creates ... e.g., **`extract_value_type`**.”
- **`uncvref`**: Not only is this name in private use for the trait, a capitalized version (**`UNCVREF`**) is in use within the Ranges TS [N4620], as well. All consulted parties considered it sufficiently clear and acceptably short. For all these reasons, this seemed the best compromise name, and so we propose it here.

4 Further discussion

A recent discussion thread on the subject of this paper yielded a number of interesting viewpoints, summarized below.¹⁰

- Peter Dimov began the thread by characterizing the trait as “obviously missing.”
- Alisdair Meredith said he is “still interested in seeing progress.” However, he also pointed out that **`std::decay`** has “cut the interest” in the proposed trait. Dimov responded that he “really can’t imagine that there exist people who use type traits and haven’t come across the need for [this trait].” Moreover, he believed that “**`std::decay`** should be used when you want decay semantics, not when you want [this trait’s] semantics.”
- Ville Voutilainen believed that “the need ... for such a trait seems surprisingly rare.” Dimov responded that “This is one of those cases where something that appears blindingly obvious to me (need and naming) turns out to be not that obvious to people.” He continued by pointing to a github search for such a trait that yielded “2,873 code results, which should be something.”
- Voutilainen admitted that seeing **`decay`** makes him “wonder whether the array and function decaying is important” but that it “usually doesn’t matter” and asks for use cases where it does matter. In follow-up replies, Dimov, Andrzej Krzemiński, and Tomasz Kamiński supplied a few, including some from the standard library. Voutilainen, although still not convinced, said he is “not going to crusade against [the proposed trait].”

¹⁰Naming concerns and off-topic remarks are not captured here.

- Dimov reiterated his point that “the lack of [this trait] combined with the presence of **decay** encourages people to do the wrong thing . . .” He later further commented, “When you read code and see **decay_t**, you don’t know whether **decay_t** has been used because decay semantics are needed, or because the intent was to merely remove references and *cv*-qualifiers.” He continued, “It can be technically correct, as in it’ll yield the right answer, but it’s still wrong because it’s the wrong word.” Tony Van Erd found “that’s enough motivation for me.”
- Nicolai Josuttis pointed out the technical importance of removing the reference before removing the *cv*-qualifiers, and the difficulty of spotting the incorrect order. Jonathan Wakely commented that “This is the most persuasive argument [he’s] seen so far.”

5 Proposed wording¹¹

5.1 Insert into [meta.type.synop] as shown:

```
namespace std {
...
    template <size_t Len, class... Types> struct aligned_union;
    template <class T> struct uncvref;
    template <class T> struct decay;
    ...
    template <size_t Len, class... Types>
        using aligned_union_t = typename aligned_union<Len, Types...>::type;
    template <class T>
        using uncvref_t = typename uncvref<T>::type;
    template <class T>
        using decay_t = typename decay<T>::type;
    ...
}
```

5.2 Between the rows specifying **aligned_union** and **decay**, insert the following new row into the “Other transformations” table in [meta.trans.other]:

<code>template <class T></code> <code>struct uncvref;</code>	The member typedef type shall name the same type as <code>remove_cv_t<remove_reference_t<T>></code> .
---	--

6 Acknowledgments

Many thanks for their thoughtful comments to Andrey Semashev and the other readers of pre-publication drafts of this paper. Special thanks to Peter Dimov for starting the recent discussion thread, and to Alisdair Meredith for drawing attention to LWG1120.

¹¹All proposed [additions](#) (there are no [deletions](#)) are relative to the post-Kona Working Draft [N4659]. Editorial notes are displayed against a `gray` background.

7 Bibliography

- [N4620] Eric Niebler and Casey Carter: “Working Draft, C++ Extensions for Ranges.” ISO/IEC JTC1/SC22/WG21 document N4620 (post-Issaquah mailing), 2016–11–27. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4620.pdf>.
- [N4659] Richard Smith: “Working Draft, Standard for Programming Language C++.” ISO/IEC JTC1/SC22/WG21 document N4659 (post-Kona mailing), 2017–03–21. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf>.

8 Document history

Version	Date	Changes
0	2017-02-01	• Published as P0550R0.
1	2017-06-11	• Updated citations and wording to conform to the post-Kona Working Draft. • Integrated mentions of LWG1120. • Added §4. • Published as P0550R1.